Common Problems

Dropbox

Handling Large Blobs

By Evan King · Updated Feb 15, 2026 · easy · Asked at: +8

Watch Video Walkthrough

Watch the author walk through the problem step-by-step

Watch Now

Understanding the Problem

What is Dropbox

Dropbox is a cloud-based file storage service that allows users to store and share files. It provides a secure and reliable way to store and access files from anywhere, on any device.

Functional Requirements

Core Requirements

1. Users should be able to upload a file from any device
2. Users should be able to download a file from any device
3. Users should be able to share a file with other users and view the files shared with them
4. Users can automatically sync files across devices

Below the line (out of scope):

- Users should be able to edit files
- Users should be able to view files without downloading them

> It's worth noting that there are related system design problems around designing Blob Storage itself. This is out of scope for this problem, but you may consider doing some

> research on your own to understand how Blob Storage works and how it's designed.

Non-Functional Requirements

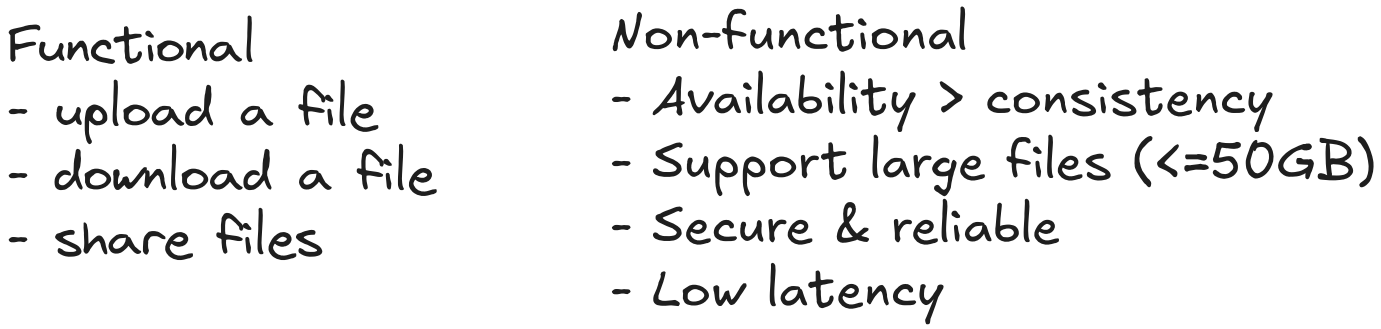
Core Requirements

1. The system should be highly available (prioritizing availability over consistency).
2. The system should support files as large as 50GB.
3. The system should be secure and reliable. We should be able to recover files if they are lost or corrupted.
4. The system should make upload, download, and sync times as fast as possible (low latency).

Below the line (out of scope):

- The system should have a storage limit per user
- The system should support file versioning
- The system should scan files for viruses and malware

Here's how it might look on your whiteboard:



Non-Functional Requirements

> Many candidates struggle with the CAP theorem trade-off for this question. Remember, you prioritize consistency over availability only if every read must receive the most recent write; otherwise, the system will break. For example, with a stock trading app, if a user buys a share of AAPL in Germany and then another user immediately tries to buy a share of AAPL in the US, you need to be sure that the first transaction has been replicated to the

> US before you can proceed. However, for a file storage system like Dropbox, it's okay if a user in Germany uploads a file and a user in the US can't see it for a few seconds.

The Set Up

Planning the Approach

Before you move on to designing the system, it's important to start by taking a moment to plan your strategy. Fortunately, for these **product design** style questions, the plan should be straightforward: build your design up sequentially, going one by one through your functional requirements. This will help you stay focused and ensure you don't get lost in the weeds as you go. Once you've satisfied the functional requirements, you'll rely on your non-functional requirements to guide you through the deep dives.

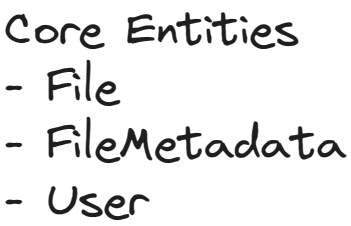
Defining the Core Entities

I like to start with a broad overview of the primary entities. At this stage, it is not necessary to know every specific column or detail. We will focus on these intricacies later when we have a clearer grasp of the system (during the high-level design). Initially, establishing these key entities will guide our thought process and lay a solid foundation as we progress towards defining the API.

For Dropbox, the primary entities are incredibly straightforward:

1. **File**: This is the raw data that users will be uploading, downloading, and sharing.
2. **FileMetadata**: This is the metadata associated with the file. It will include information like the file's name, size, mime type, and the user who uploaded it.
3. **User**: The user of our system.

In the actual interview, this can be as simple as a short list like this. Just make sure you talk through the entities with your interviewer to ensure you are on the same page.



Defining the Core Entities

> As you move onto the design, your objective is simple: create a system that meets all functional and non-functional requirements. To do this, I recommend you start by satisfying the functional requirements and then layer in the non-functional requirements afterward. This will help you stay focused and ensure you don't get lost in the weeds as you go.

API or System Interface

The API is the primary interface that users will interact with. It's important to define the API early on, as it will guide your high-level design. We just need to define an endpoint for each of our functional requirements.

Starting with uploading a file, we might have an endpoint like this:

```
POST /files
Request:
{
  File,
  FileMetadata
}
```

To download a file, our endpoint can be:

```
GET /files/{fileId} -> File & FileMetadata
```

> Be aware that your APIs may change or evolve as you progress. In this case, our upload and download APIs actually evolve significantly as we weigh the trade-offs of various approaches in our high-level design (more on this later). You can proactively communicate this to your interviewer by saying, "I am going to outline some simple APIs, but may come back and improve them as we delve deeper into the design."

To share a file, we might have an endpoint like this:

```
POST /files/{fileId}/share
Request:
{
  User[] // The users to share the file with
}
```

Lastly, we need a way for clients to query for changes to files on the remote server. This way we know which files need to be synced to the local device.

```
GET /files/changes?since={timestamp} -> ChangeEvent[]
```

Each `ChangeEvent` includes the `fileId`, the type of change (created, updated, deleted), and the updated metadata. By passing a `since` timestamp, the client can efficiently fetch only the changes that have occurred since its last sync.

> With each of these requests, the user information will be passed in the headers (either via session token or JWT). This is a common pattern for APIs and is a good way to ensure that the user is authenticated and authorized to perform the action while preserving security. You should avoid passing user information in the request body, as this can be easily manipulated by the client.

High-Level Design

1) Users should be able to upload a file from any device

The main requirement for a system like Dropbox is to allow users to upload files. When it comes to storing a file, we need to consider two things:

1. Where do we store the file contents (the raw bytes)?
2. Where do we store the file metadata?

For the metadata, we can use a **NoSQL database** like DynamoDB. **DynamoDB** is a fully managed NoSQL database hosted by AWS. Our metadata is loosely structured, with few relations and the main query pattern being to fetch files by user. This makes DynamoDB a solid choice, but don't get too caught up in making the right choice here in your interview. The reality is a SQL database like PostgreSQL would work just as well for this use case. Learn more about how to choose the right database (and why it may not matter), **here**.

Our schema will be a simple document and can start with something like this:

```
{
  "id": "123",
  "name": "file.txt",
  "size": 1000,
  "mimeType": "text/plain",
  "uploadedBy": "user1"
}
```

As for how we store the file itself, we have a few options. Let's take a look at the trade-offs of each.

Bad Solution: Upload File to a Single Server

Good Solution: Store File in Blob Storage

Great Solution: Upload File Directly to Blob Storage

Pattern: Handling Large Blobs

The direct upload approach using presigned URLs is a classic example of handling large file transfers efficiently. This pattern of bypassing application servers for data transfer, using signed URLs for security, and implementing chunked uploads for reliability appears across many distributed systems that handle substantial file uploads and downloads.

Learn This Pattern

2) Users should be able to download a file from any device

The next step is making sure users can download their saved files. Just like with uploads, there are a few different ways to approach this.

Bad Solution: Download through File Server

Good Solution: Download from Blob Storage

Great Solution: Download from CDN

3) Users should be able to share a file with other users

To round out the functional requirements, we need to support sharing files with other users. We will implement this similarly to Google Drive, where you just need to enter the email address of the user you want to share the file with. We can assume users are already authenticated.

The main consideration here in an interview is how you can make this process fast and efficient. Let's break it down.

Bad Solution: Add a Sharelist to Metadata

Good Solution: Caching to speed up fetching the Sharelist

Great Solution: Create a separate table for shares

4) Users can automatically sync files across devices

Last up, we need to make sure that files are automatically synced across different devices. At a high level, this works by keeping a copy of a particular file on each client device (locally) and also in remote storage (i.e., the "cloud"). As such, there are two directions we need to sync in:

1. Local -> Remote
2. Remote -> Local

Local -> Remote

When a user updates a file on their local machine, we need to sync these changes with the remote server. We consider the remote server to be the source of truth, so it's important that we get it consistent as soon as possible so that other local devices can know when there are changes they should pull in.

To do this, we need a client-side sync agent that:

1. Monitors the local Dropbox folder for changes using OS-specific file system events (like FileSystemWatcher on Windows or FSEvents on macOS)
2. When it detects a change, it queues the modified file for upload locally
3. It then uses our upload API to send the changes to the server along with updated metadata
4. Conflicts are resolved using a "last write wins" strategy - meaning if two users edit the same file, the most recent edit will be the one that's saved

> Versioning is out of scope for this write-up, but note that you would typically not overwrite the only file. Instead, you'd add a new file (or at least the new chunks) and update a version number and pointer on the metadata.

Remote -> Local

For the other direction, each client needs to know when changes happen on the remote server so they can pull those changes down.

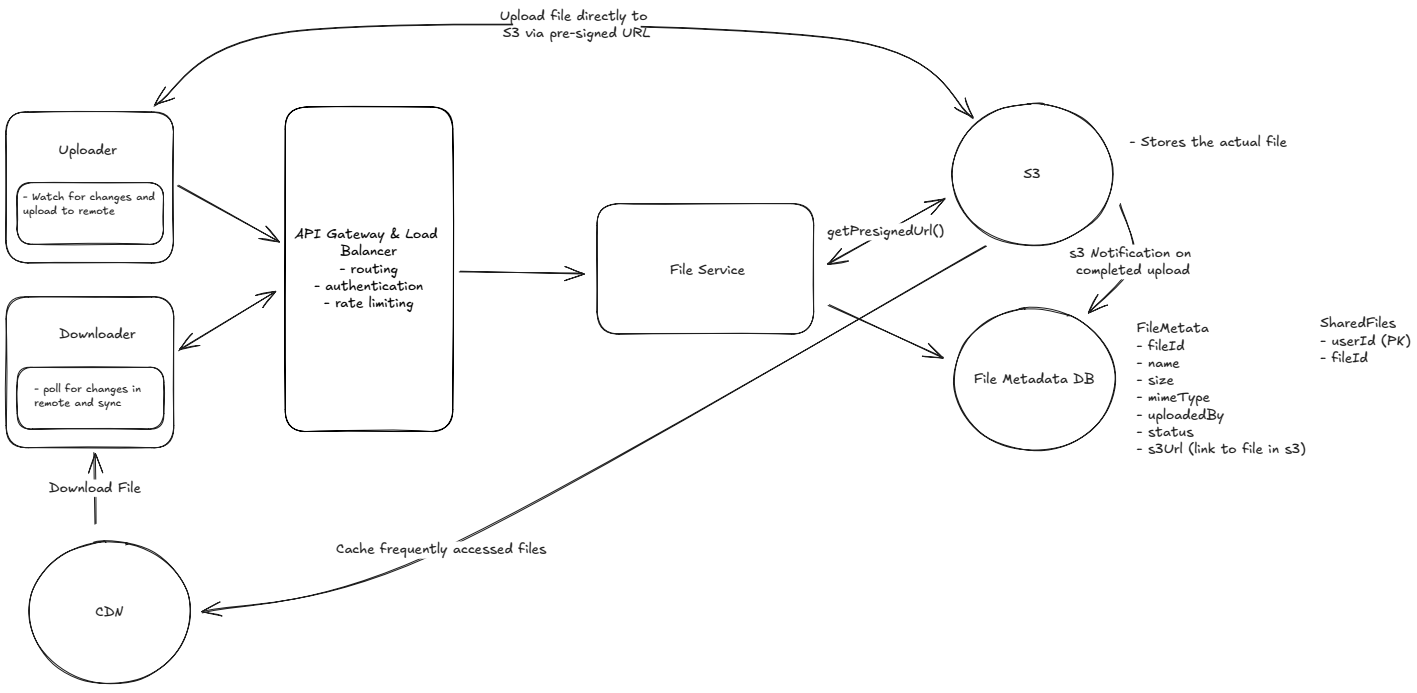
There are two main approaches we could take:

1. **Polling**: The client periodically asks the server "has anything changed since my last sync?" The server would query the DB to see if any files that this user is watching has a updatedAt timestamp that is newer than the last time they synced. This is simple but can be slow to detect changes and wastes bandwidth if nothing has changed.
2. **WebSocket or SSE**: The server maintains an open connection with each client and pushes notifications when changes occur. This is more complex but provides real-time updates.

For Dropbox, we can use a hybrid approach. Each client maintains a single WebSocket (or SSE) connection to the server, not one per file, but one per device/session. Through this connection, the server pushes change notifications for files the user has access to. The hybrid part comes in with how we handle reliability:

- **Active notification**: The server pushes change events through the WebSocket connection in real-time as they happen. This gives us near-instant sync for any file change.
- **Periodic polling as a safety net**: WebSocket connections can drop, and messages can be missed. To handle this, the client also periodically polls the server (e.g., every few minutes) using `GET /files/changes?since={timestamp}` to catch any changes it might have missed. This ensures eventual consistency even if the WebSocket connection was temporarily interrupted.

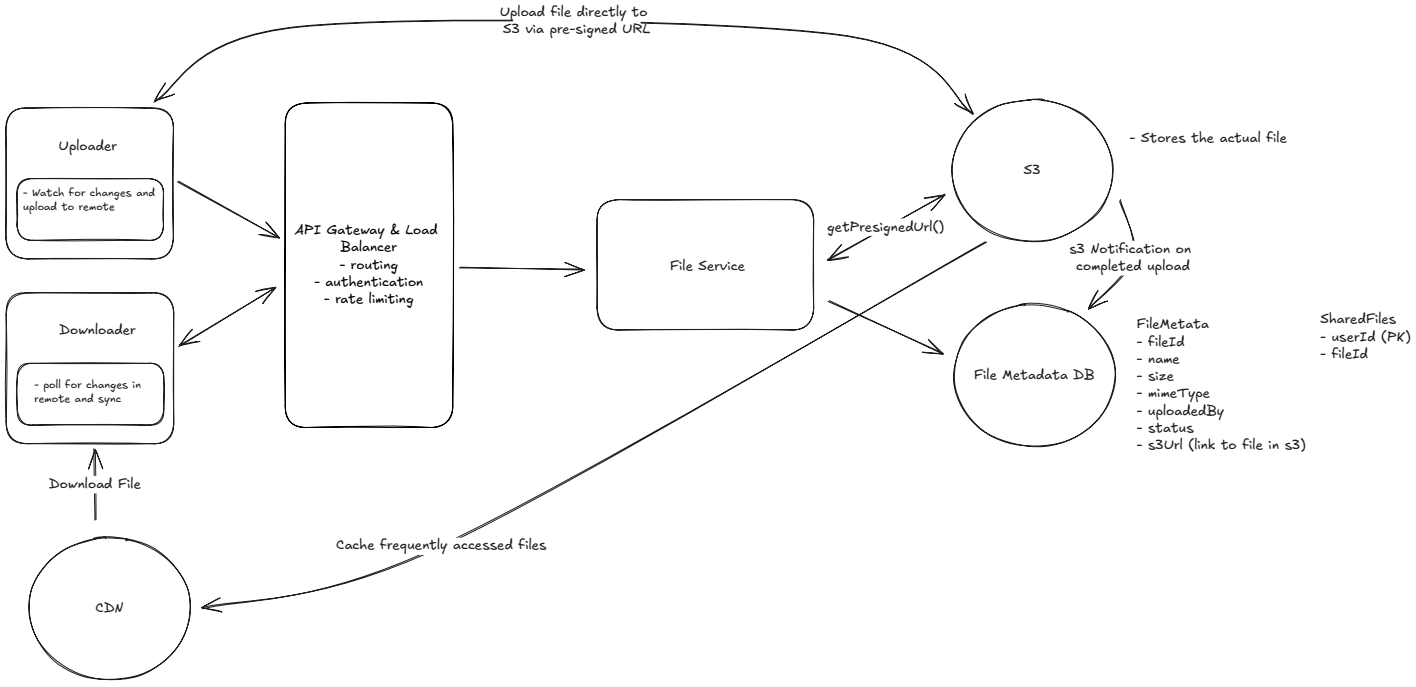
This hybrid approach gives us the best of both worlds — real-time updates through push notifications, with polling as a reliable fallback to guarantee no changes are ever lost.



Sync

Tying it all together

Let's take a step back and look at our system as a whole. At this point, we have a simple design that satisfies all of our functional requirements.



Final

- **Uploader**: This is the client that uploads the file. It could be a web browser, a mobile app, or a desktop app. It is also responsible for proactively identifying local changes and pushing the updates to remote storage.
- **Downloader**: This is the client that downloads the file. Of course, this can be the same client as the uploader, but it doesn't have to be. We separate them in our design for clarity. It is also responsible for determining when a file it has locally has changed on the remote server and downloading these changes.
- **LB & API Gateway**: This is the load balancer and API Gateway that sits in front of our application servers. It's responsible for routing requests to the appropriate server and handling things like SSL termination, rate limiting, and request validation.
- **File Service**: The file service is responsible for reading and writing file metadata in the database and generating presigned URLs using the S3 SDK. Generating a presigned URL is a purely local operation, meaning the service uses its AWS credentials to cryptographically

sign a URL without making any call to S3. The file service doesn't handle file uploads or downloads directly; it's the control plane that coordinates between the client and S3.

- **File Metadata DB**: This is where we store metadata about the files. This includes things like the file name, size, MIME type, and the user who uploaded the file. We also store a shared files table here that maps files to users who have access to them. We use this table to enforce permissions when a user tries to download a file.
- **S3**: This is where the files are actually stored. We upload files directly to S3 using presigned URLs generated by the file service.
- **CDN**: This is a content delivery network (like CloudFront) that caches files close to the user to reduce latency. For downloads, instead of giving users a direct S3 presigned URL, the file service generates a CDN signed URL. The CDN fetches the file from S3 on the first request (cache miss) and serves it from the edge on subsequent requests (cache hit). This means users download from the nearest CDN edge location rather than from the S3 region directly.

Potential Deep Dives

1) How can you support large files?

The first thing you should consider when thinking about large files is the user experience. There are two key insights that should stick out and ultimately guide your design:

1. **Progress Indicator**: Users should be able to see the progress of their upload so that they know it's working and how long it will take.
2. **Resumable Uploads**: Users should be able to pause and resume uploads. If they lose their internet connection or close the browser, they should be able to pick up where they left off rather than re-uploading the 49GB that may have already been uploaded before the interruption.

This is, in some sense, the meat of the problem and where I usually end up spending the most time with candidates in a real interview.

Before we go deep on solutions, let's take a moment to acknowledge the limitations that come with uploading a large file via a single POST request.

- **Timeouts**: Web servers and clients typically have timeout settings to prevent indefinite waiting for a response. A single POST request for a 50GB file could easily exceed these timeouts. In fact, this may be an appropriate time to do some quick math in the interview. If we have a 50GB file and an internet connection of 100Mbps, how long will it take to upload the file? `50GB * 8 bits/byte / 100Mbps = 4000 seconds` then `4000 seconds / 60 seconds/minute / 60 minutes/hour = 1.11 hours`. That's a long time to wait without any response from the server.
- **Browser and Server Limitation**: In most cases, it's not even possible to upload a 50GB file via a single POST request due to limitations configured in the browser or on the server. Both browsers and web servers often impose limits on the size of a request payload. While raw web servers like Apache and NGINX can be configured to accept large payloads, most modern services like Amazon API Gateway have hard limits that are much lower and cannot be increased. This is just 10MB in the case of Amazon API Gateway which we are using in our design.
- **Network Interruptions**: Large files are more susceptible to network interruptions. If a user is uploading a 50GB file and their internet connection drops, they will have to start the upload from scratch.
- **User Experience**: Users are effectively blind to the progress of their upload. They have no idea how long it will take or if it's even working.

To address these limitations, we can use a technique called "chunking" to break the file into smaller pieces and upload them one at a time (or in parallel, depending on network bandwidth). Chunking needs to be done on the client so that the file can be broken into pieces before it is sent to the server (or S3 in our case). A very common mistake candidates make is to chunk the file on the server, which effectively defeats the purpose since you still upload the entire file at once to get it on the server in the first place. When we chunk, we typically break the file into 5-10 MB pieces, but this can be adjusted based on the network conditions and the size of the file.

With chunks, it's rather straightforward for us to show a progress indicator to the user. We can simply track the progress of each chunk and update the progress bar as each chunk is successfully uploaded. This provides a much better user experience than the user simply staring at a spinning wheel for an hour.

The next question is: **how will we handle resumable uploads?** We need to keep track of which chunks have been uploaded and which haven't. We can do this by saving the state of the upload in the database, specifically in our `FileMetadata` table. Let's update the `FileMetadata` schema to include a `chunks` field.

```
{
  "id": "123",
  "name": "file.txt",
  "size": 1000,
  "mimeType": "text/plain",
  "uploadedBy": "user1",
  "status": "uploading",
  "chunks": [
    {
      "id": "chunk1",
      "status": "uploaded"
    },
    {
      "id": "chunk2",
      "status": "uploading"
    },
    {
      "id": "chunk3",
      "status": "not-uploaded"
    }
  ]
}
```

When the user resumes the upload, we can check the `chunks` field to see which chunks have been uploaded and which haven't. We can then start uploading the chunks that haven't been uploaded yet. This way, the user doesn't have to start the upload from scratch if they lose their internet connection or close the browser.

But how should we ensure this `chunks` field is kept in sync with the actual chunks that have been uploaded?

There are two approaches we can take:

Good Solution: Update based on client PATCH request

Great Solution: Server-Side Chunk Verification

Next, let's talk about how to uniquely identify a file and a chunk. When you try to resume an upload, the very first question that should be asked is: (1) Have I tried to upload this file before? and (2) If yes, which chunks have I already uploaded? To answer the first question, we cannot naively rely on the file name. This is because two different users (or even the same user) could upload files with the same name. Instead, we need to rely on a unique identifier that is derived from the file's content. This is called a fingerprint.

A **fingerprint** is a mathematical calculation that generates a unique hash value based on the content of the file. This hash value, often created using cryptographic hash functions like SHA-256, serves as a robust and unique identifier for the file's content regardless of its name or the source of the upload. By computing this fingerprint, we can efficiently determine whether the file has been uploaded before (deduplication) and whether an in-progress upload can be resumed.

Note that the fingerprint identifies the file *content*, not the file *record*. Two different users uploading the same file would produce the same fingerprint. In practice, the `fileId` in your metadata table should be a unique identifier (like a UUID), while the fingerprint is stored as a separate field used for deduplication and resumability checks.

For resumable uploads, the process involves not only fingerprinting the entire file but also generating fingerprints for each individual chunk. This chunk-level fingerprinting allows the system to precisely identify which parts of the file have already been transmitted.

Taking a step back, we can tie it all together. Here is what will happen when a user uploads a large file:

1. The client will chunk the file into 5-10MB pieces and calculate a fingerprint for each chunk. It will also calculate a fingerprint for the entire file, which is used to check for duplicates and resumability.
2. The client will send a request to check if a file with the same fingerprint already exists for this user. If it does and has a status of "uploading", the client can resume the upload by fetching the existing chunk statuses.
3. If the file does not exist, the client will POST a request to initiate a multipart upload. The backend will call S3's CreateMultipartUpload API to get an `uploadId`, generate presigned

URLs for each part, save the file metadata in the `FileMetadata` table with a status of "uploading", and return the `uploadId` along with presigned URLs for each chunk.

4. The client will then upload each chunk to S3 using its corresponding presigned URL (each part requires its own presigned URL with the `uploadId` and `partNumber`). After each chunk is uploaded, the client sends a PATCH request to our backend with the chunk status and ETag. Our backend can then verify the chunk uploads with S3's ListParts API before updating the `chunks` field in the `FileMetadata` table to mark the chunk as "uploaded".
5. Once all chunks in our `chunks` array are marked as "uploaded", the backend calls S3's **CompleteMultipartUpload** API with the list of part numbers and ETags. This tells S3 to assemble all the parts into a single object. Only after S3 confirms successful assembly does the backend update the `FileMetadata` table to mark the file as "uploaded".

All throughout this process, the client is responsible for keeping track of the progress of the upload and updating the user interface accordingly so the user knows how far in they are and how much longer it will take.

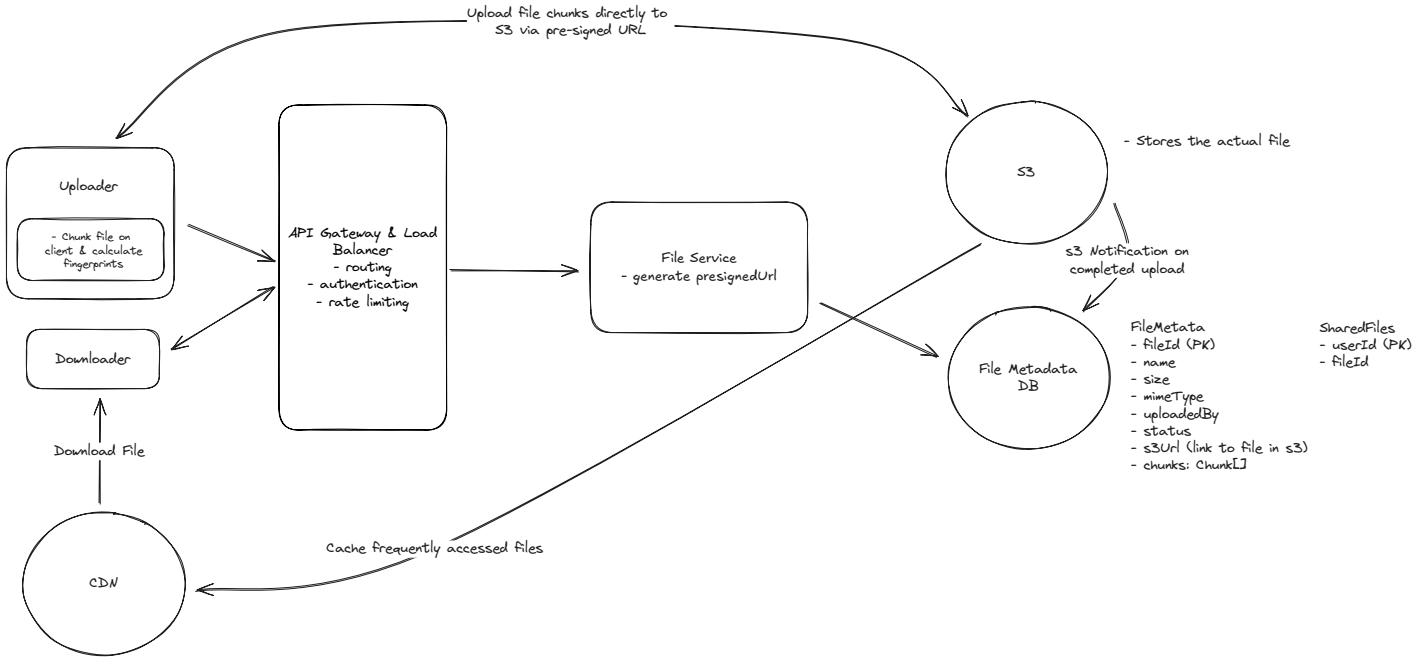
The approach we just described is not novel. In fact, this is a problem that has been solved by cloud storage providers like Amazon S3. They have a feature called Multipart Upload that allows you to upload large objects in parts. This is exactly what we just described. The client breaks the file into parts and uploads each part to S3. S3 then combines the parts into a single object. They even provide a handy JavaScript SDK which will handle all of the chunking and uploading for you.

With S3 multipart uploads, event notifications only trigger when the entire multipart upload is completed (when all parts are assembled), not for individual part uploads. For

Show More

You might wonder whether chunked uploads mean we also need chunked downloads. Not quite. With S3 multipart upload, once `CompleteMultipartUpload` is called, S3 assembles all the parts into a single object. From that point on, downloads work like any normal file. The client gets a single presigned URL (or CDN signed URL) and downloads the complete file.

> For very large files, S3 and HTTP natively support Range requests, which let the client download different byte ranges in parallel or resume an interrupted download without starting over. The client doesn't need to know anything about the original chunk boundaries.



How can you support large files?

2) How can we make uploads, downloads, and syncing as fast as possible?

We've already touched on a few ways to speed up both download and upload respectively, but there is still more we can do to make the system as fast as possible. To recap, for download we used a CDN to cache the file closer to the user. This made it so that the file doesn't have to travel as far to get to the user, reducing latency and speeding up download times. For upload, chunking, beyond being useful for resumable uploads, also plays a significant role in speeding up the upload process. While bandwidth is fixed (put another way, the pipe is only so big), we can use chunking to make the most of the bandwidth we have. By sending multiple chunks in parallel, and utilizing adaptive chunk sizes based on network conditions, we can maximize the use of available bandwidth. The same chunking approach can be used for syncing files. When a file changes, we only need to sync the chunks that actually changed rather than the entire file, making syncing much faster.

One subtlety worth calling out is that if you use fixed-size chunks (e.g., every 5MB), inserting a single byte near the beginning of the file shifts all subsequent chunk boundaries, causing *every* chunk after the edit to produce a different fingerprint. This makes delta sync nearly useless. The solution is Content-Defined Chunking (CDC), where chunk boundaries are determined by the file's content using a rolling hash (like Rabin fingerprinting). With CDC, a small edit only affects the chunks immediately surrounding the change, the vast majority of chunks remain identical. This is how systems like Dropbox actually achieve efficient delta sync in practice.

Beyond that which we've already discussed, **we can also utilize compression to speed up both uploads and downloads**. Compression reduces the size of the file, which means fewer bytes need to be transferred. Since we're uploading directly to S3, compression happens entirely on the client side: the client compresses the file before uploading, and the compressed data is stored in S3 as-is. When downloading, the client decompresses the file after retrieving it. This keeps our backend out of the data path while still benefiting from reduced transfer sizes.

We'll need to be smart about when we compress though. Compression is only useful if the speed gained from transferring fewer bytes outweighs the time it takes to compress and decompress the file. For some file types, particularly media files like images and videos, the compression ratio is so low that it's not worth the time it takes to compress and decompress the file. If you take a `.png` off your computer right now and compress it, you'll be lucky to have decreased the file size by more than a few percent -- so it's not worth it. For text files, on the other hand, the compression ratio is much higher and, depending on network conditions, it may very well be worth it. A 5GB text file could compress down to 1GB or even less depending on the content.

In the end, you'll want to implement logic on the client that decides whether or not to compress the file before uploading it based on the file type, size, and network conditions.

There are a number of compression algorithms that you can use to compress files. The most common are Gzip, Brotli, and Zstandard. Each of these algorithms has its own tradeoffs in terms of compression ratio and speed. Gzip is the most widely used and has broad support everywhere. Brotli generally achieves better compression ratios than Gzip (especially for text), and is supported by all modern browsers (Chrome, Firefox, Edge,

> Safari). Zstandard (zstd) offers an excellent balance of speed and compression ratio — it compresses and decompresses significantly faster than Gzip at comparable ratios, and can be tuned across a wide range of speed/ratio tradeoffs. For a system like Dropbox
>
> Show More

3) How can you ensure file security?

Security is a critical aspect of any file storage system. We need to ensure that files are secure and only accessible to authorized users.

1. **Encryption in Transit**: Sure, to most candidates, this is a no-brainer. We should use HTTPS to encrypt the data as it's transferred between the client and the server. This is a standard practice and is supported by all modern web browsers.
2. **Encryption at Rest**: We should also encrypt the files when they are stored in S3. This is a feature of S3 and is easy to enable. When a file is uploaded to S3, we can specify that it should be encrypted. S3 will then encrypt the file using a unique key and store the key separately from the file. This way, even if someone gains access to the file, they won't be able to decrypt it without the key. You can learn more about S3 encryption **here**.
3. **Access Control**: Our `shareList` or separate share table/cache is our basic ACL. As discussed earlier, we make sure that we share download links only with authorized users.

But what happens if an authorized user shares a download link with an unauthorized user? For example, an authorized user may, intentionally or unintentionally, post a download link to a public forum or social media and we need to make sure that unauthorized users cannot download the file.

This is where those signed URLs we talked about early come back into play. When a user requests a download link, we generate a signed URL that is only valid for a short period of time (e.g. 5 minutes). This signed URL is then sent to the user, who can use it to download the file. It's worth noting that signed URLs are bearer tokens - anyone with a valid, unexpired URL can download the file. The short expiration window limits the exposure, but doesn't fully prevent sharing. For higher security scenarios, you could add additional restrictions like IP binding or require the signed URL to be used in conjunction with authentication cookies.

They also work with modern CDNs like CloudFront and are a feature of S3. Here is how:

1. Generation: A signed URL is generated on the server, including a signature that typically incorporates the URL path, an expiration timestamp, and possibly other restrictions (like IP address). For CloudFront, this signature is created using the content provider's private key.
2. Distribution: The signed URL is distributed to an authorized user, who can use it to access the specified resource directly from the CDN.
3. Validation: When the CDN receives a request with a signed URL, it verifies the signature using the corresponding public key (which was registered with CloudFront), checks the expiration timestamp and any other restrictions. If the signature is valid and the URL has not expired, the CDN serves the requested content. If not, it denies access.

What is Expected at Each Level?

Ok, that was a lot. You may be thinking, “how much of that is actually required from me in an interview?” Let’s break it down.

Mid-level

Breadth vs. Depth: A mid-level candidate will be mostly focused on breadth (80% vs 20%). You should be able to craft a high-level design that meets the functional requirements you've defined, but many of the components will be abstractions with which you only have surface-level familiarity.

Probing the Basics: Your interviewer will spend some time probing the basics to confirm that you know what each component in your system does. For example, if you add an API Gateway, expect that they may ask you what it does and how it works (at a high level). In short, the interviewer is not taking anything for granted with respect to your knowledge.

Mixture of Driving and Taking the Backseat: You should drive the early stages of the interview in particular, but the interviewer doesn’t expect that you are able to proactively recognize problems in your design with high precision. Because of this, it’s reasonable that they will take over and drive the later stages of the interview while probing your design.

The Bar for Dropbox: For this question, an E4 candidate will have clearly defined the API endpoints and data model, landed on a high-level design that is functional for all of uploading, downloading, and sharing. I don't expect candidates to know about pre-signed URLs or uploading/downloading to/from S3 directly, or to immediately know about chunking. However, I expect that when I ask probing questions like, "You're uploading the file twice right now, how can we avoid that?" or "How can you show a user's progress while allowing them to resume an upload?" that they can reason through the problem and come to a solution via some back and forth.

Senior

Depth of Expertise: As a senior candidate, expectations shift towards more in-depth knowledge — about 60% breadth and 40% depth. This means you should be able to go into technical details in areas where you have hands-on experience. It's crucial that you demonstrate a deep understanding of key concepts and technologies relevant to the task at hand.

Advanced System Design: You should be familiar with advanced system design principles. For example, knowing how to utilize blob storage for large files, or how to implement a CDN for faster downloads. You should be able to discuss the trade-offs involved in different design choices and justify your decisions based on your experience.

Articulating Architectural Decisions: You should be able to clearly articulate the pros and cons of different architectural choices, especially how they impact scalability, performance, and maintainability. You justify your decisions and explain the trade-offs involved in your design choices.

Problem-Solving and Proactivity: You should demonstrate strong problem-solving skills and a proactive approach. This includes anticipating potential challenges in your designs and suggesting improvements. You need to be adept at identifying and addressing bottlenecks, optimizing performance, and ensuring system reliability.

The Bar for Dropbox: For this question, E5 candidates are expected to quickly go through the initial high-level design so that they can spend time discussing, in detail, how to handle uploading large files, in particular. I expect them to be more proactive here than mid-level candidates, thinking through several options and arriving at a reasonable solution. While not

strictly required, many candidates will have experience with file uploads and can speak directly about certain APIs (like multipart upload) and how they work.

Staff+

Emphasis on Depth: As a staff+ candidate, the expectation is a deep dive into the nuances of system design — I'm looking for about 40% breadth and 60% depth in your understanding. This level is all about demonstrating that, while you may not have solved this particular problem before, you have solved enough problems in the real world to be able to confidently design a solution backed by your experience.

You should know which technologies to use, not just in theory but in practice, and be able to draw from your past experiences to explain how they'd be applied to solve specific problems effectively. The interviewer knows you know the small stuff (REST API, data normalization, etc) so you can breeze through that at a high level so you have time to get into what is interesting.

High Degree of Proactivity: At this level, an exceptional degree of proactivity is expected. You should be able to identify and solve issues independently, demonstrating a strong ability to recognize and address the core challenges in system design. This involves not just responding to problems as they arise but anticipating them and implementing preemptive solutions. Your interviewer should intervene only to focus, not to steer.

Practical Application of Technology: You should be well-versed in the practical application of various technologies. Your experience should guide the conversation, showing a clear understanding of how different tools and systems can be configured in real-world scenarios to meet specific requirements.

Complex Problem-Solving and Decision-Making: Your problem-solving skills should be top-notch. This means not only being able to tackle complex technical challenges but also making informed decisions that consider various factors such as scalability, performance, reliability, and maintenance.

Advanced System Design and Scalability: Your approach to system design should be advanced, focusing on scalability and reliability, especially under high load conditions. This includes a thorough understanding of distributed systems, load balancing, caching strategies, and other advanced concepts necessary for building robust, scalable systems.

The Bar for Dropbox: For a staff-level candidate, expectations are high regarding the depth and quality of solutions, especially for the complex scenarios discussed earlier. Exceptional candidates delve deeply into each of the topics mentioned above and may even steer the conversation in a different direction, focusing extensively on a topic they find particularly interesting or relevant. They are also expected to possess a solid understanding of the trade-offs between various solutions and to be able to articulate them clearly, treating the interviewer as a peer.